AI Usage Log – Sprint 2

Project: SOEN 341 – CareerConnect (Job Search and Application Tracking Platform)

Team member: [Your Name]

Sprint 2 feature: Job Search and Job Listing Interface

AI tool used: Claude Code (Claude desktop app, Code tab), model Claude Opus 5.5

Date of AI session: October 29, 2026

Summary of AI-Assisted Activities

#	Task	Decision
1	Plan the Job Search feature and approve changes to existing code	Accepted
2	Implement the Job Search backend (database, API endpoints, tests, docs)	Accepted
3	Implement the Job Search and Job Details pages and link them into the site	Accepted

Task 1: Plan the Job Search feature and approve changes to existing code

Date: October 29, 2026

Task ID/Title	Task 1: Plan the Job Search feature and approve changes to existing code
Purpose of AI Use	Project planning, system design and code review (deciding how the new feature connects to the existing codebase).
Chat Link or Prompt/Response	Local Claude Code session "Job search features" (no shareable link). Prompt and response in Appendix, A.1.
AI-Suggested Content	• Read the Sprint 2 brief and the existing code (server.js, database migrations, the home, profile and resume pages, the API docs, teammates' Sprint 2 branches) before writing anything. • Found that the existing test in test/database.test.js expected exactly one migration, so any new migration would break it (Jack's application-tracking branch included). • Asked for approval before touching existing files, proposing four changes: (1) wire the job routes and CORS into server.js, (2) make the migration-count test count the migration files, (3) turn the "Job Search – Soon" item on the home page into a real link and add a "05 Job search" card, (4) add a Job Search section to docs/sprint_2/backend_api.md.

	• Recommended doing the work on a new branch, feature/sprint2-job-search, following docs/workflow.md.
Validation	• I reviewed each proposed change to existing files before approving it, as my prompt required. • Checked the plan against the three Sprint 2 "Job Search and Job Listing Interface" requirements. • Checked the branch name against the team's branching strategy in docs/workflow.md.
Decision	Accepted: all four changes to existing files and the new feature branch were approved.
Reflection	Asking the AI to get approval before changing existing code kept me in control of shared files. It also spotted a test that would have broken for other teammates. The migration-count problem should be shared with Jack before he merges.
Responsible Person	[Your Name]

Task 2: Implement the Job Search backend (database, API endpoints, tests, docs)

Date: October 29, 2026

Task ID/Title	Task 2: Implement the Job Search backend (database, API endpoints, tests, docs)
Purpose of AI Use	Code generation, test generation and documentation.
Chat Link or Prompt/Response	Local Claude Code session "Job search features" (no shareable link). Prompt and response in Appendix, A.1–A.2.
AI-Suggested Content	• db/migrations/003_jobs.sql: a jobs table plus 10 sample jobs from made-up companies, because recruiters cannot post jobs until the Sprint 3 recruiter dashboard exists. Named 003 so it does not collide with Jack's 002_applications.sql. • routes/jobs.js: three public endpoints: GET /api/jobs (search by keyword, location, category and job type, newest first), GET /api/jobs/categories (filter options) and GET /api/jobs/:id (job details). • Search treats characters like % as plain text. The AI first wrote LIKE ... ESCAPE, found pg-mem (the in-memory test database) does not support ESCAPE, and switched to a substring match (strpos/position). • test/jobs.test.js: tests for the endpoints. It retries the migration once, because Node runs test files in parallel and both files can migrate the shared CI database at the same time.

	• server.js (3 lines) and test/database.test.js (migration count), as approved in Task 1. • docs/sprint_2/backend_api.md: a new Job Search section.
Validation	• npm test: both test files pass on pg-mem. • The AI caught and fixed a garbled line in its own first version of jobs.test.js. • Not yet tested against real PostgreSQL because Docker was not running; the CI pipeline will run it on push. • To do before merging: peer review through a pull request.
Decision	Accepted (code on branch feature/sprint2-job-search, not yet committed at the end of the chat). Update this if the team changes it during PR review.
Reflection	Useful: the AI produced working endpoints and tests fast, and it adapted when the test database lacked a SQL feature. The sample jobs are a stopgap until recruiters can post jobs in Sprint 3. Running the tests against real PostgreSQL in CI is still needed.
Responsible Person	[Your Name]

Task 3: Implement the Job Search and Job Details pages and link them into the site

Date: October 29, 2026

Task ID/Title	Task 3: Implement the Job Search and Job Details pages and link them into the site
Purpose of AI Use	Code generation and UI/UX design (front end that matches the existing site style).
Chat Link or Prompt/Response	Local Claude Code session "Job search features" (no shareable link). Prompt and response in Appendix, A.1 and A.3.
AI-Suggested Content	• frontend/jobs.html and js/jobs.js: job listings as white cards on the burgundy background, with keyword, location, category and job-type filters that can be combined. The current search is kept in the URL, so it survives a reload and can be shared. • frontend/job.html and js/job-details.js: full description, requirements and a job summary; "Back to job search" returns to the same results. • frontend/css/jobs.css built from the same palette and patterns as auth.css and resume.css. • index.html: the sidebar item is now a real link, plus a new "05 Job search" card (approved in Task 1).

	• The Apply button is a placeholder ("available once application tracking is connected"). It already gathers jobId, jobTitle and company, the shape Jack's application branch expects.
Validation	• The AI tested the pages in a browser against a local demo server: all 10 sample jobs load across origins (CORS works), keyword + location search, URL syncing, the details page, the back link, the Apply message, an invalid job id, an empty search, and the mobile layout (no horizontal scrolling). • It investigated a dark band at the bottom of one screenshot and confirmed it was a screenshot artifact, not a CSS bug. • Manual review of the pages against the existing site style.
Decision	Accepted. Known gaps left on purpose: Apply is a placeholder until Jack's application API is merged and login saves a token, and the "Browse Jobs" link on profile.html still points to # because that file was not approved for changes.
Reflection	The pages matched the site's look with little rework, and the browser testing covered edge cases I might have skipped. Follow-ups: connect Apply after the application-tracking merge, and update the profile page's "Browse Jobs" link.
Responsible Person	[Your Name]

Appendix – Prompts and AI Responses

The AI conversation was held in a local Claude Code session, which has no shareable link, so the prompts and the relevant AI responses are reproduced below. Actions the AI took (reading files, running commands, editing code) are summarized in brackets.

A.1 – Initial prompt and approval of changes

Prompt

```
@SOEN341_CareerConnect_Sprint2.docx
Implement the job search features to the project. Respect the style of the website and if
you have to bring changes to existing code please let me know before doing any changes. Your
implementation of job search should connect with the rest of the project.
```

AI response (approval questions)

[AI read the Sprint 2 brief, the backend (server.js, db/, migrations, API docs), the frontend pages and styles, teammates' Sprint 2 branches, and ran the existing tests.]

Question 1: Which changes to existing files do you approve? (The first two are needed for job search to work end to end.)

- server.js wiring + CORS: add 2 lines inside createApp, app.use('/api/jobs', cors()) and require('./routes/jobs')(app, pool), placed before the error handler. Nothing else in server.js changes.
- Migration count in test: test/database.test.js asserts schema_migrations rowCount === 1. Any new migration breaks it, so change it to count the .sql files in db/migrations (Jack's branch will hit this too).
- Home page link: index.html, turn the "Job Search – Soon" sidebar item into a real link to frontend/jobs.html and add a 05 "Job search" destination card.
- API docs: docs/sprint_2/backend_api.md, add a Job Listings section and remove the line saying job endpoints aren't included yet.

Question 2: Where should I do the work?

- New feature branch (Recommended): create feature/sprint2-job-search from main, following docs/workflow.md. Nothing is committed until you ask.
- Stay on main: edit files directly on main, uncommitted.

My answer

```
Approved: server.js wiring + CORS, Migration count in test, Home page link, API docs
Branch: New feature branch (Recommended)
```

A.2 – Backend implementation (progress notes from the AI)

"Approved, so I'm on the new feature/sprint2-job-search branch. Baseline tests pass. Next I'm writing the migration and the jobs route module."

[AI wrote db/migrations/003_jobs.sql and routes/jobs.js, then edited server.js and test/database.test.js as approved.]

"Now the jobs test, so I can see whether pg-mem handles ILIKE and the seed data."

"That test file has one garbled line. I'll replace it with a proper company-name match check."

"pg-mem doesn't support ESCAPE. Docker exists but its daemon isn't running. Trying a substring match that needs no wildcards at all (strpos / position)."

[AI rewrote the search to use a substring match and re-ran the tests.]

A.3 – Frontend implementation and browser testing (progress notes from the AI)

[AI wrote frontend/jobs.html, js/jobs.js, job.html, js/job-details.js.]

"Now the stylesheet, built from the same palette and patterns as auth.css and resume.css."

"Pages are done. Next are the approved home page and API docs edits." [AI edited index.html and backend_api.md, started a temporary demo server.]

"The page loads across origins (CORS works), shows all 10 sample jobs, and matches the site's look. Now I'm testing keyword plus location search, and then the details page."

"Search, URL syncing and the result cards all work. There's a dark band at the bottom of the viewport, though. Checking whether it comes from my CSS or the shared auth.css background."

"The dark band was a screenshot artifact; it's gone now. The details page renders well. Next I'm checking the back to search link, the Apply message, a bad id, an empty search and the mobile layout."

"Everything behaves as expected, including the mobile layout." [AI checked for horizontal scrolling, stopped the temporary servers and re-ran the tests.]

A.4 – Final AI response

Job search is built and connected to the rest of the site. It's on the new branch feature/sprint2-job-search, and nothing is committed yet. Both test files pass using pg-mem (the in-memory database the existing tests use). It has not run against real PostgreSQL because Docker wasn't running; CI will do that on push. I also tried it in the browser: search, filters, the details page, error messages and the mobile layout all worked.

What it does (the three Sprint 2 bullets):

- Job listings: jobs.html shows open jobs as white cards on the burgundy background, newest first. Sprint 3's recruiter dashboard can't post jobs yet, so 003_jobs.sql adds 10 sample jobs from made-up companies.
- Search and filters: you can search by keyword (title, company or description), location, category and job type, and combine them. The current search is kept in the URL, so it survives a reload and can be shared.
- Job details: job.html shows the full description, the requirements and a summary of the job. "Back to job search" returns to the same results.

Backend: routes/jobs.js adds three public endpoints: search (GET /api/jobs), filter options (GET /api/jobs/categories) and job details (GET /api/jobs/:id). Searches treat characters like % as plain text. Tests are in jobs.test.js.

Changes to existing files (the four you approved):

- server.js: 3 lines that load the job routes and allow the frontend to call them from Live Server.
- database.test.js: the migration-count check now counts the migration files instead of expecting exactly 1.
- index.html: the sidebar's "Job Search – Soon" item is now a real link, plus a new "05 Job search" card.
- backend_api.md: a new Job Search section.

Things to know:

- Apply button: it's a placeholder that shows "available once application tracking is connected", like the resume page does. It already gathers jobId, jobTitle and company, which is the shape Jack's application branch expects. Applying won't actually work until Jack's API is merged and login saves a token (auth.js doesn't call the backend yet).
- Jack's branch: his migration is 002_applications.sql and mine is 003_jobs.sql, so they won't collide. He also asserts the migration count is 1, so he'll need the same one-line fix when he merges.
- Tests on a shared database: Node runs test files in parallel, so in CI both files can try to migrate the same database at once. jobs.test.js retries the migration once to handle this.
- Profile page: profile.html has a "Browse Jobs" link that still points to #. I left it alone because you didn't approve that file.